

LAPORAN PRAKTIKUM
BAHASA PEMROGRAMAN 2
MODUL 10

Dosen pengampu : Yulyanto, S.Kom., M.TI.



Disusun oleh :

Nama : Muhammad Rizal Nurfirdaus
NIM : 20230810088
Jadwal : Rabu, 14:40 – 16:15
Kelas : TINF-C-2023-04

PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS ILMU KOMPUTER
UNIVERSITAS KUNINGAN

2025

DAFTAR ISI

DAFTAR ISI	ii
BAB I	1
PRETEST	1
BAB II	2
PRAKTIKUM	2
1. Untuk kode programnya ada di link Github Ini : https://github.com/MuhammadRizalNurfirdaus/BP2/tree/3d2c1133320a14e4726e88f20c18a879ff5b2992/Modul_9/Anggota	2
BAB III	4
POSTTEST	4
1. Apa yang disimpulkan tentang DAO dan MVC pada Jawa!	4
BAB IV	5
TUGAS	5
1. Buka kembali file Project Praktikum Modul 1–8.	5
2. Lengkapi tabel yang pernah dibuat sebelumnya di DBMS MySQL pada Modul 5, dengan struktur Physical Data Model sebagai berikut: <i>(Bisa dimodifikasi sesuai kebutuhan)</i>	5
3. Setelah selesai melengkapi tabel tersebut, Buatlah Project Aplikasi berbasis Java yang dilengkapi dengan:	5
BAB V	12
KESIMPULAN	12

BAB I
PRETEST

BAB II

PRAKTIKUM

1. Untuk kode programnya ada di link Github Ini :
https://github.com/MuhammadRizalNurfirdaus/BP2/tree/3d2c1133320a14e4726e88f20c18a879ff5b2992/Modul_9/Anggota

DATA ANGGOTA HIMA

ID Id Otomatis

NAMA

ALAMAT

NO. TELP

Pencarian Berdasarkan Nama :

ID	Nama	Alamat	NoPe
1	Muhammad Rizal ...	Desa Sidaraja	083112341234

DATA ANGGOTA HIMA

ID Id Otomatis

NAMA

ALAMAT

NO. TELP

Pencarian Berdasarkan Nama :

ID	Nama	Alamat	NoPe
1	Muhammad Rizal ...	Desa Sidaraja	083112341234

Message

Data Berhasil Disimpan

ID	Nama	Alamat	NoPe
1	Muhammad Rizal ...	Desa Sidaraja	083112341234
2	Udin	Cigugur	08421734216

Analisis : Berdasarkan tampilan antarmuka aplikasi pada gambar, ini merupakan program berbasis Java Swing yang digunakan untuk mengelola data anggota HIMA (Himpunan Mahasiswa). Program ini memungkinkan pengguna untuk melakukan operasi CRUD (Create, Read, Update, Delete) terhadap data anggota, seperti memasukkan nama, alamat, dan nomor telepon. Tombol INSERT berfungsi untuk menyimpan data baru ke dalam tabel, sedangkan UPDATE, DELETE, dan RESET digunakan untuk mengedit, menghapus, dan mengosongkan form input. Notifikasi berupa dialog "Data Berhasil Disimpan" juga menunjukkan bahwa aplikasi menggunakan JOptionPane untuk memberikan umpan balik kepada pengguna.

Selain itu, aplikasi ini juga memiliki fitur pencarian berdasarkan nama untuk memudahkan dalam menemukan data tertentu di dalam tabel. Data yang ditampilkan dalam tabel mencerminkan input pengguna yang telah berhasil disimpan, termasuk ID yang otomatis ter-generate. Tampilan dan struktur komponen menunjukkan penggunaan JTable untuk menampilkan data, serta JTextField, JButton, dan JLabel sebagai elemen form. Seluruh interaksi terjadi di dalam satu jendela, yang kemungkinan besar merupakan turunan dari JFrame. Aplikasi ini cocok digunakan dalam skala kecil-menengah untuk pengelolaan data sederhana.

BAB III

POSTTEST

1. Apa yang disimpulkan tentang DAO dan MVC pada Jawa!
DAO (Data Access Object) dan **MVC (Model-View-Controller)** pada Java merupakan dua konsep penting yang sering digunakan untuk memisahkan logika aplikasi agar lebih terstruktur dan mudah dikelola.
 - a. **DAO (Data Access Object)** adalah pola desain yang digunakan untuk memisahkan logika akses data (seperti koneksi ke database, eksekusi query, dll) dari logika bisnis aplikasi. Dalam konteks Java, DAO biasanya berupa kelas-kelas yang berisi metode untuk melakukan operasi CRUD (Create, Read, Update, Delete) terhadap database. Tujuannya adalah agar jika nanti ingin mengganti database atau teknologi penyimpanan data, kita hanya perlu mengubah bagian DAO-nya saja, tanpa menyentuh logika lainnya.
 - b. **MVC (Model-View-Controller)** adalah pola arsitektur yang membagi aplikasi menjadi tiga bagian utama:
 - **Model:** berisi data dan logika bisnis (termasuk DAO biasanya berada di sini),
 - **View:** berisi tampilan antarmuka pengguna (GUI, seperti Java Swing/JSP),
 - **Controller:** menangani alur data antara Model dan View, serta mengatur logika interaksi pengguna.

Dengan menggunakan MVC di Java, pengembangan aplikasi menjadi lebih modular, memudahkan debugging, testing, dan pemeliharaan aplikasi dalam jangka panjang.

BAB IV

TUGAS

1. Buka kembali file Project Praktikum Modul 1–8.
2. Lengkapi tabel yang pernah dibuat sebelumnya di DBMS MySQL pada Modul 5, dengan struktur Physical Data Model sebagai berikut: *(Bisa dimodifikasi sesuai kebutuhan)*
(Terdapat gambar ERD dengan tabel berikut ini:)

tbl_barang

tbl_penjualan

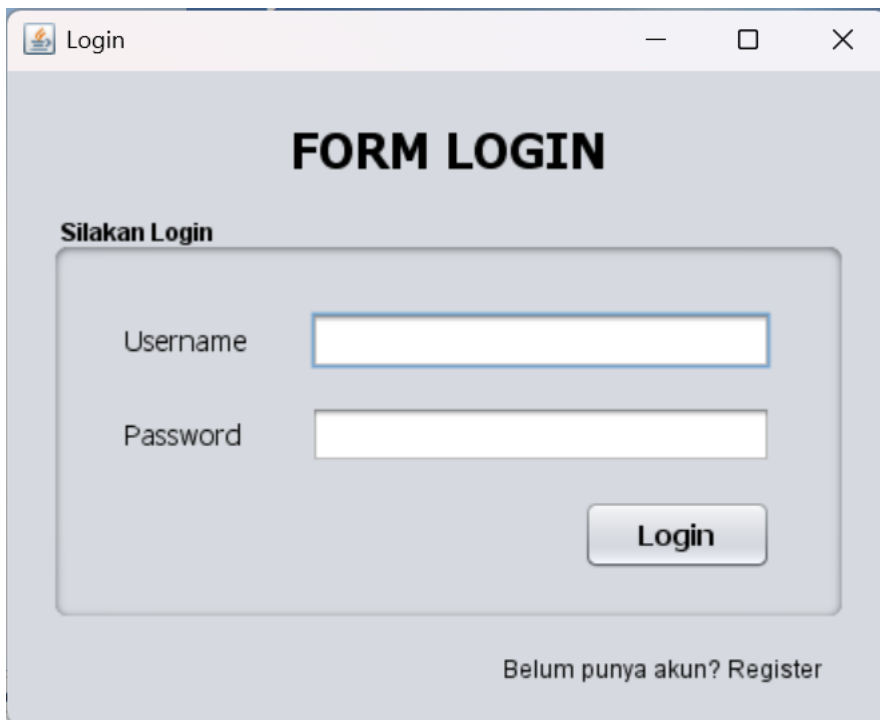
tbl_pembelian

tbl_user

tbl_level

tbl_supplier

3. Setelah selesai melengkapi tabel tersebut, Buatlah Project Aplikasi berbasis Java yang dilengkapi dengan:
 - 1) Form Login
 - 2) Form Utama dengan Menu
 - 3) Form CRUD untuk masing-masing datastore
 - 4) Buat juga Report dari masing-masing datastore tersebut.



Login

FORM LOGIN

Silakan Login

Username

Password

Login

Belum punya akun? Register

Form Registrasi

REGISTRASI USER BARU

Form Registrasi User

ID User

Nama Lengkap

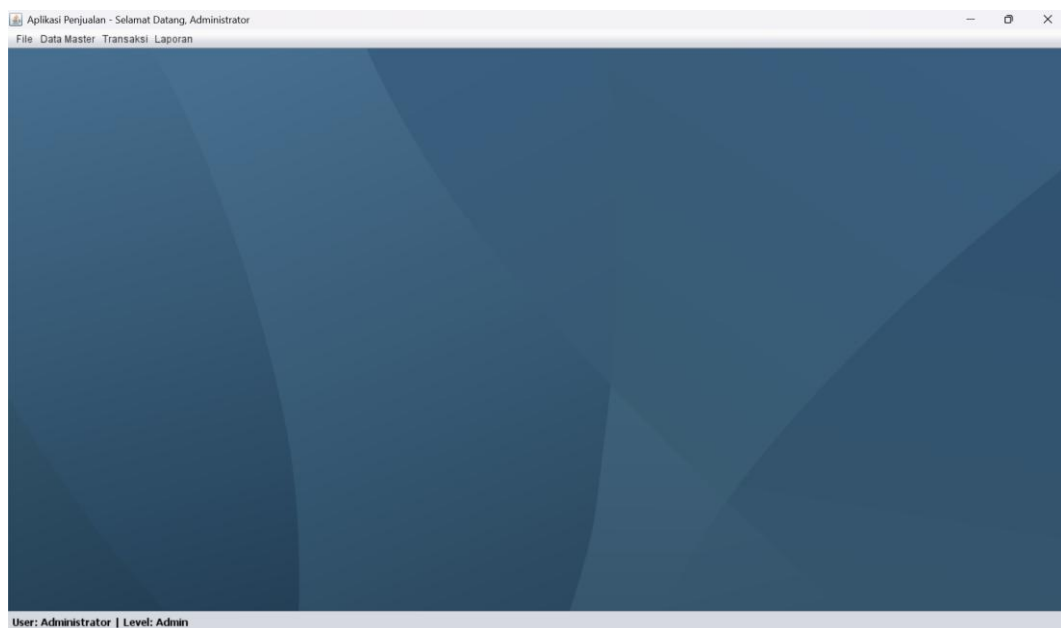
Jenis Kelamin ☒ Laki-laki ☐ Perempuan

No. HP

Username

Password

[Kembali ke Login](#) [Register](#)



Aplikasi Penjualan - Selamat Datang, Administrator
File Data Master Transaksi Laporan

Data User

DATA MASTER USER

ID User Username

Nama User Password

Jenis Kelamin ☒ Laki-laki ☐ Perempuan Level

No. HP

Cari User (Nama)

ID User	Nama User	Jenis Kelamin	No. HP	Username	Level
USER001	Administrator	Laki-laki	081234567890	admin	Admin

User: Administrator | Level: Admin

Aplikasi Penjualan - Selamat Datang, Administrator
File Data Master Transaksi Laporan

Data Barang

DATA MASTER BARANG

Kode Barang

Nama Barang

Harga Jual

Harga Beli

Stok

Cari Barang (Nama)

Kode Barang	Nama Barang	Harga Jual	Harga Beli	Stok
1	Susu	18000	12000	10

Aplikasi Penjualan - Selamat Datang, Administrator
File Data Master Transaksi Laporan

DATA MASTER SUPPLIER

Kode Supplier

Nama Supplier

No. HP

Alamat

Cari Supplier (Nama)

Kode Supplier	Nama Supplier	No. HP	Alamat
1	Agus	084632751623	Cigugur

Stok

Aplikasi Penjualan - Selamat Datang, Administrator
File Data Master Transaksi Laporan

TRANSAKSI PENJUALAN

No. Nota Kasir Bertugas: Administrator

Pilih Barang Stok Saat Ini

Harga Jual

Jumlah

Total Harga

Riwayat Penjualan Terakhir

No. Nota	Tanggal	Nama Barang	Jumlah	Total Jual	Kasir
NT1750156271473	17-06-2025	Susu	2	36000	Administrator

Aplikasi Penjualan - Selamat Datang, Administrator
File Data Master Transaksi Laporan

Transaksi Pembelian

TRANSAKSI PEMBELIAN (STOK MASUK)

Kode Pembelian: User Bertugas: Administrator

Pilih Barang:

Harga Beli:

Jumlah:

Total Harga:

Riwayat Pembelian Terakhir

Kode Pembelian	Tanggal	Nama Barang	Jumlah	Total Beli	User
PB1750154239053	17-06-2025	Susu	1	0	Administrator
PB1750154466532	17-06-2025	Susu	1	12000	Administrator

Aplikasi Penjualan - Selamat Datang, Administrator
File Data Master Transaksi Laporan

Transaksi Penjualan

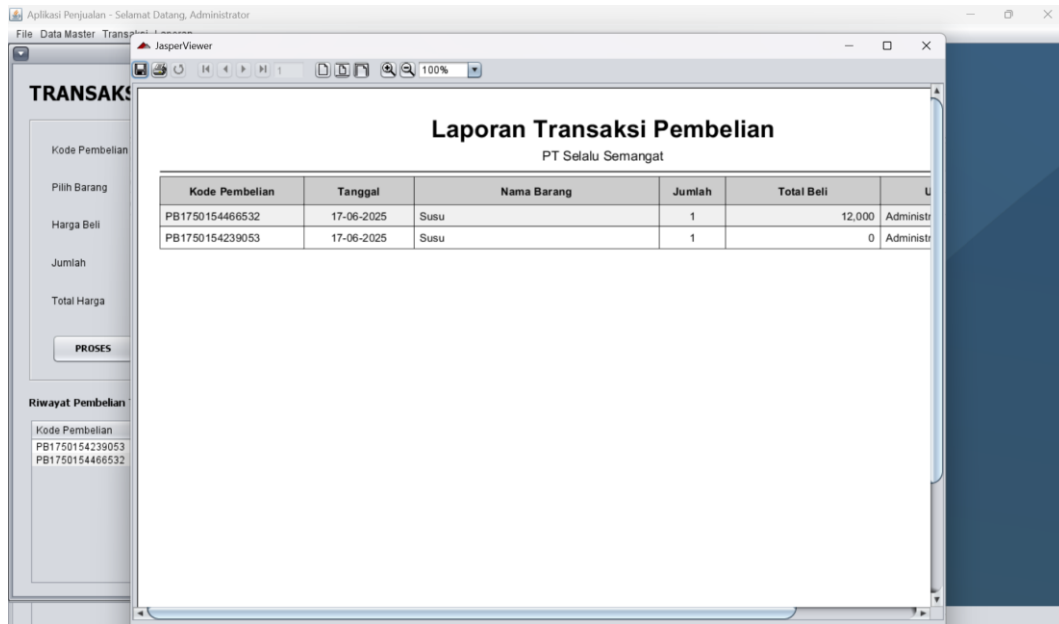
Laporan Transaksi Penjualan

PT Selalu Semangat

No. Nota	Tanggal	Nama Barang	Jumlah	Total Jual	User
NT1750156271473	17-06-2025	Susu	2	36,000	Administrator

Riwayat Pembelian

Kode Pembelian:



Analisis : Setelah memahami keseluruhan kode dari proyek AplikasiPenjualan ini, saya bisa menyimpulkan bahwa fondasi utamanya adalah arsitektur yang sangat kokoh dan terstruktur dengan baik, yaitu perpaduan antara **Model-View-Controller (MVC)** dengan pola desain **Data Access Object (DAO)**. Arsitektur ini membagi aplikasi menjadi tiga bagian utama yang logis: *View* sebagai lapisan antarmuka yang dilihat pengguna (seperti FormLogin dan FormBarang), *Model* sebagai representasi data murni (seperti kelas User.java dan Barang.java), dan *Controller* sebagai otak yang menjadi jembatan antara keduanya. Kerennya lagi, arsitektur ini diperkuat dengan pola DAO, yang secara cerdas memisahkan logika bisnis di Controller dari urusan teknis database. Lapisan DAO (seperti DaoUser dan DaoBarang) secara eksklusif

bertanggung jawab atas semua perintah SQL, membuat aplikasi menjadi jauh lebih rapi dan mudah dikelola.

Alur kerja aplikasi ini sangat terstruktur dan logis, yang memastikan setiap komponen punya tugasnya masing-masing dan tidak saling tumpang tindih. Sebagai contoh, ketika seorang pengguna mengisi data di FormBarang dan menekan tombol 'Insert', alurnya berjalan mulus: FormBarang (View) tidak langsung berbicara ke database, melainkan hanya memberitahu ControllerBarang bahwa ada aksi yang perlu dilakukan. Controller kemudian mengambil data dari View, membungkusnya ke dalam objek Barang (Model), dan meneruskan objek ini ke DaoBarang (DAO) untuk dieksekusi ke dalam database melalui perintah SQL INSERT. Setelah selesai, Controller kembali mengambil alih tugas untuk memberi tahu View agar me-refresh tabel dan mengosongkan form. Alur yang elegan ini terjadi pada semua fitur CRUD di seluruh aplikasi, menciptakan sebuah sistem yang konsisten dan mudah dilacak.

Kekuatan terbesar dari arsitektur yang digunakan ini adalah *separation of concerns* atau pemisahan tugas yang jelas, yang memberikan dua keuntungan besar: kemudahan perawatan (maintainability) dan kemudahan pengembangan (scalability). Jika suatu saat kita ingin mengganti database dari MySQL ke PostgreSQL, kita hanya perlu mengubah lapisan DAO saja; lapisan View dan Controller tidak perlu disentuh sama sekali. Kemudahan pengembangannya terlihat jelas saat kita menambahkan modul baru seperti 'Data Supplier' setelah 'Data Barang' selesai. Pola yang digunakan sama persis, kita hanya perlu membuat satu set file MVC+DAO baru untuk Supplier, membuatnya sangat efisien dan mengurangi kemungkinan kesalahan. Terakhir, integrasi dengan JasperReports untuk fitur pelaporan menjadi pelengkap yang sempurna, mengubah data mentah menjadi informasi yang berguna dan siap cetak untuk pengambilan keputusan. Secara keseluruhan, proyek ini bukan hanya sekadar aplikasi CRUD biasa, melainkan sebuah sistem mini yang kokoh, mudah dikembangkan, dan siap untuk ditambahkan fitur-fitur yang lebih kompleks di kemudian hari.

BAB V

KESIMPULAN

Awalnya, proyek ini terlihat seperti tugas biasa untuk membuat aplikasi desktop, namun seiring berjalannya waktu, ini menjadi sebuah perjalanan arsitektural yang mendalam. Kunci utama dari keberhasilan dan kerapian proyek ini adalah keputusan sadar untuk tidak membuat kode secara asal-asalan, melainkan mengadopsi arsitektur **MVC (Model-View-Controller) yang dipadukan dengan pola DAO (Data Access Object)**. Ini seperti membangun sebuah gedung pencakar langit; kita tidak langsung memasang jendela di lantai 50, melainkan membangun fondasi yang kokoh terlebih dahulu. MVC bertindak sebagai cetak biru utama, memisahkan secara tegas antara tampilan antarmuka (View), logika bisnis (Controller), dan struktur data (Model). Diperkuat dengan DAO, kita menciptakan sebuah lapisan "spesialis" yang tugasnya hanya satu: berinteraksi dengan database. Hasilnya? Sebuah sistem yang sangat terorganisir, di mana setiap komponen tahu persis tugasnya masing-masing tanpa mengganggu yang lain.

Perjalanan pengerjaan proyek ini terasa sangat sistematis berkat arsitektur yang solid. Kita memulainya dari gerbang utama, yaitu sistem **Login dan Registrasi**, yang tidak hanya mengautentikasi pengguna tetapi juga mengatur hak akses menu berdasarkan level. Ini adalah fondasi keamanan aplikasi. Setelah itu, kita masuk ke inti, yaitu membangun modul-modul master data untuk **User, Barang, dan Supplier**. Di sini, keindahan pola yang kita gunakan benar-benar terasa; kita bisa "menduplikasi" pola kerja dari satu modul ke modul lainnya dengan sangat efisien. Puncaknya adalah saat kita mengerjakan modul **Transaksi Pembelian dan Penjualan**. Ini bukan lagi sekadar CRUD biasa. Di sini kita belajar tentang konsep *database transaction*, di mana dua perintah SQL—misalnya, INSERT data penjualan dan UPDATE stok barang—harus berhasil bersamaan. Jika salah satu gagal, semuanya akan dibatalkan (rollback), sehingga integritas data kita tetap terjaga. Ini adalah lompatan besar dari sekadar aplikasi data entry menjadi sebuah sistem yang andal.

Pada akhirnya, proyek ini jauh lebih dari sekadar aplikasi yang selesai. Fitur **pelaporan dengan JasperReports** menjadi sentuhan akhir yang profesional, mengubah tumpukan data di database menjadi informasi visual yang berarti dan siap dicetak. Setiap error yang kita hadapi, mulai dari `NullPointerException`, file .form yang korup, hingga `Uncompilable source code`, bukanlah sebuah kegagalan, melainkan pelajaran berharga tentang pentingnya ketelitian, sinkronisasi, dan *defensive programming*. Bagi saya, Aplikasi Penjualan ini adalah bukti nyata bahwa dengan perencanaan arsitektur yang matang dan pemahaman alur kerja yang jelas, kita bisa membangun sebuah sistem yang tidak hanya berfungsi dengan baik, tetapi juga kokoh, mudah dikelola, dan siap untuk dikembangkan lebih jauh lagi di masa depan. Sebuah portofolio yang membanggakan.

Link Github Untuk Kode Lengkapnya :

https://github.com/MuhammadRizalNurfirdaus/BP2/tree/916b7b9f848eb267073a6335d9d2980ac358e3d1/Modul_9